COMSATS University Islamabad, Attock campus

Department of Computer Science

Program: BS-SE

Sp24-BSE-058

Sumbal

Terminal exam

Information Security

16-December-2025

Ma'am Ambreen Gul

Question# 01:

Q1: Secure Email Communication System

1. **RSA Key Generation:**

```
import math

# Two small prime numbers (for demo)
p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

# Choose public key e
e = 17

# Compute private key d
def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d

d = mod_inverse(e, phi)

print("RSA Public Key  (e, n):", (e, n))
print("RSA Private Key (d, n):", (d, n))

```

2. **RSA Encryption & Decryption:**

```
# ----------------------------
# RSA ENCRYPTION / DECRYPTION
# ----------------------------

# Message (convert to number)
message = 65
print("Encryption and Decryption using RSA")
print("\nOriginal Message:", message)

# Encryption
ciphertext = pow(message, e, n)
print("Encrypted Message:", ciphertext)
```

```
# Decryption
decrypted_message = pow(ciphertext, d, n)
print("Decrypted Message:", decrypted_message)
```

3.ElGamal Digital Signature & Verification:

```
# ----------------------------
# EL GAMAL DIGITAL SIGNATURE
# ----------------------------
import random

# Public parameters
p = 467       # large prime
g = 2         # generator

# Private key
x = random.randint(1, p-2)

# Public key
y = pow(g, x, p)

print("\nElGamal Public Key (p, g, y):", (p, g, y))
print("ElGamal Private Key x:", x)

# Message to sign
message = 123
hash_value = message   # simplified hash for demo

# Choose random k
k = random.randint(1, p-2)
while math.gcd(k, p-1) != 1:
    k = random.randint(1, p-2)

# Signature generation
r = pow(g, k, p)
k_inv = pow(k, -1, p-1)
s = (k_inv * (hash_value - x * r)) % (p-1)

print("\nSignature (r, s):", (r, s))
```

. **Signature verification:**

```
# ----------------------------
# SIGNATURE VERIFICATION
# ----------------------------

left = pow(g, hash_value, p)
right = (pow(y, r, p) * pow(r, s, p)) % p

print("\nVerification:")
print("Left Side :", left)
print("Right Side:", right)

if left == right:
    print("Signature is VALID")
else:
    print("Signature is INVALID")
```

Output of code:

```
p\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\hp
y-2025.16.0-win32-x64\bundled\libs\debugpy\launcher' '62170' '--'
e\py.py'
RSA Public Key  (e, n): (17, 3233)
RSA Private Key (d, n): (2753, 3233)
Encryption and Decryption using RSA

Original Message: 65
Encrypted Message: 2790
Decrypted Message: 65
```

```
ElGamal Public Key (p, g, y): (467, 2, 365)
ElGamal Private Key x: 36

Signature (r, s): (370, 193)

Verification:
Left Side : 125
Right Side: 125
Signature is VALID
PS C:\Users\hp\Desktop\python workspace>
```

Output of the code for demo email:

```
n.exe' 'c:\Users\hp\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundled\libs\debugpy\launcher' '62402' '--' 'c:\Users\hp\Desktop\python workspace\py.py'
RSA Public Key : (17, 3233)
RSA Private Key: (2753, 3233)

Original Email: Hello Alice, this is a secure email.

Encrypted Email: [3000, 1313, 745, 745, 2185, 1992, 2790, 745, 3179, 281, 1313, 678, 1992, 884, 2170, 3179, 1230, 1992, 3179, 1230, 1992, 1632, 1992, 1230, 1313, 281, 2160, 2412, 1313, 1992, 1313, 2271, 1632, 3179, 745, 2825]

Decrypted Email: Hello Alice, this is a secure email.

ElGamal Public Key: (467, 2, 294)
ElGamal Private Key: 281

Signature (r, s): (136, 266)

Signature Verification:
Signature is VALID
PS C:\Users\hp\Desktop\python workspace>
```

Question# 02:

Q2. Project-Based Security Analysis

Project: Secure Communication Between Two Participants Using ElGamal

1. Justification of Security Method

ElGamal is suitable for secure communication because:

- It provides **strong public-key security**
- Based on the **Discrete Logarithm Problem**, which is computationally hard
- Supports **both encryption and digital signatures**
- Ensures **confidentiality and authentication**
- More secure than basic symmetric encryption (like DES) for key exchange
- Resistant to brute-force attacks with large key sizes

Compared to older methods like DES or simple RSA-only systems, ElGamal provides **better randomness and signature support**, making it ideal for secure participant communication.

---

2. Possible Vulnerability or Weakness

Weakness: Poor Random Number (k) Selection

If the random value k used during signature generation is:

- Reused
- Predictable
- Too small

An attacker can:

- Recover the private key x
- Forge digital signatures
- Impersonate a legitimate user

This compromises **authentication and integrity**.

---

3. Suggested Improvement

Improvement: Use Secure Hash + Secure Random Generator

Solution:

- Use a cryptographically secure random number generator for k
- Hash the message using **SHA-256** before signing
- Combine ElGamal with **ECC-based key exchange** for smaller key sizes and higher security

How it works:

- Secure randomness prevents private key leakage
- Hashing ensures message integrity
- ECC improves efficiency and security level

This enhancement significantly strengthens the system against attacks.
